ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

FACULTAD DE INGERNIERÍA EN ELECTRICIDAD Y COMPUTACIÓN
INGENIERÍA DE SOFTWARE I
ESPECIFICACIÓN DE PROYECTO TERCERA EVALUACIÓN

Objetivos Específicos

1. Organizar el desarrollo de un sistema de software mediante la conformación de equipos de trabajo, considerando roles y responsabilidades.
2. Aplicar normas y buenas prácticas de ingeniería de requerimientos para la generación de especificaciones de requerimientos de software para un sistema de mediana complejidad.

Objetivos de Aprendizaje

2. Diseñar, implementar y evaluar una solución basada en computación para cumplir con un conjunto dado de requisitos de computación en el contexto de la disciplina del programa.

Prerrequisito

1. **El documento de especificación de requerimientos completo** del sistema de software aprobado para la asignatura. Este prerrequisito (**incluyendo el prototipo de alta fidelidad** [9]) debe constar entre los entregables. La evaluación del prerrequisito **se constituye en un factor entre 0 y 1 en la evaluación de este subsiguiente avance del proyecto**.
2. **El reporte completo de la documentación de planificación y diseño del sistema de software aprobado para la asignatura.**

Descripción

El documento debe contener los elementos descritos en las secciones que se presentan a continuación. **En todo momento, el diseño del sistema debe ser consistente con el documento de especificación de requerimientos y el prototipo del sistema**.

La aceptación del contenido del documento por parte del cliente debe constar en un acta de aceptación debidamente firmada.

Sección A

Realizar el análisis de la arquitectura propuesta para su sistema de software por medio del método ATAM [21, 22].

Sección B

Realizar la medición del tamaño de su sistema de software por medio de la metodología COSMIC [23].

Entregables

Como plantilla del documento, considere la plantilla LaTeX y guías especificadas en la referencia “Writing guidelines and templates” [10] de ETS pero utilizando el logo de ESPOL.

1. El reporte del avance en el desarrollo del sistema de software (**en formato pdf**).
   a. Debe incluir una lista de los integrantes del equipo en la primera página y tabla de contenido, índice de tablas e índice de figuras en las subsiguientes páginas.
   b. Diseño de alto nivel y modelos legibles del sistema de software haciendo uso de los diagramas UML vistos en clase. Cada diagrama debe estar en una página y la imagen debe tener buena resolución. Indicar la herramienta en la que se creó los modelos del sistema.
   c. Puede incluir cualquier información que soporte su trabajo con relación a los elementos referidos en la rúbrica de evaluación.
   d. Cronograma y documentación de riesgos.
   e. Un acta de conformidad con el contenido firmado por el representante del cliente.
   f. Análisis de la arquitectura propuesta para su sistema de software.

g. Medición del tamaño funcional de su sistema de software.

2. El documento de especificación de requerimientos que al igual que el parcial anterior incluya:
   a. Un prototipo de alta fidelidad del sistema de software aprobado. Adicionalmente, capturas de pantalla del prototipo y su flujo de ventanas.
   b. Un acta de conformidad con el contenido firmado por el representante del cliente.
3. El proyecto correspondiente a la herramienta en la cual se modeló el diseño del sistema (por ejemplo, Visual Paradigm).
4. Cada estudiante debe enviar la co-evaluación del desempeño de los otros integrantes del equipo de trabajo completando la **tarea individual** "Co-evaluación Proyecto".

Rúbrica de Calificación

Descripción	**Valor**
Sección A	
Análisis arquitectónico con ATAM	50
Sección B	
Medición del software con COSMIC	50
Total	100
Penalidades	
Ausencia de acta de aceptación firmada por el cliente	-100
No subir un entregable de acuerdo con lo requerido (máximo -30)	-10
Cada error gramatical u ortográfico (penalidad máxima: -20)	-02
Cada error de notación UML (penalidad máxima: -30)	-03
Organización de elementos que dificulte la legibilidad de un diagrama (penalidad máxima: -50)	-10
No subir coevaluación en la correspondiente tarea individual (afectación individual)	-100

Late Submission Policy

Delay (§)	Penalty (Ω)
1 hour or less	loss of 10%
1 to 6 hours	loss of 20%
6 to 12 hours	loss of 30%
Over 12 hours:	loss of 100%

(§) every clock hour counts including weekends or holidays
(Ω) automatic and non-negotiable penalty

Referencias

1. Robertson S. and Robertson J, Mastering the Requirements Process: Getting Requirements Right
2. Pressman and Maxim, "Software Engineering"
3. Ian Somerville, "Software Engineering"
4. IEEE Recommended Practice for Software Requirements Specifications
5. Systems and software engineering - Life cycle processes – Requirements engineering
6. The IEEE Guide to the Software Engineering Body of Knowledge – SWEBOK
7. MockupScreens, http://www.mockupscreens.com
8. Perdita Stevens, Using UML

9. Jaye Hannah, “What’s the Difference Between a Wireframe, a Prototype, and a Mockup?”, https://careerfoundry.com/en/blog/ux-design/difference-between-wireframes-prototypes-mockups/
10. ETS, “Writing guidelines and templates”, https://www.etsmtl.ca/en/studies/Guidelines-and-templates
11. Medvidovic, N., & Taylor, R. N. (2000). A classification and comparison framework for software architecture description languages. IEEE Transactions on software engineering, 26(1), 70-93.
12. Visual Paradigm, “Tutorials”, https://online.visual-paradigm.com/diagrams/tutorials/
13. Visual Paradigm, “Learning Guides”, https://www.visual-paradigm.com/guide/
14. Visual Paradigm, “Software Design Handbook”, https://www.visual-paradigm.com/learning/handbooks/software-design-handbook/
15. Kirill Fakhroutdinov, “The Unified Modeling Language”, https://www.uml-diagrams.org
16. Bizzdesign, “UML modeling”, https://support.bizzdesign.com/display/knowledge/UML+modeling
17. Jakob Jenkov, “Exception Handling Strategy – Overview”, https://jenkov.com/tutorials/exception-handling-strategies/overview.html
18. Common Weakness Enumeration, “Bad Coding Practices”, https://cwe.mitre.org/data/definitions/1006.html
19. Dzone, “9 Best Practices to Handle Exceptions in Java”, https://dzone.com/articles/9-best-practices-to-handle-exceptions-in-java
20. Baeldung, “Exception Handling in Java”, https://www.baeldung.com/java-exceptions
21. Kazman et al., “ATAM: Method for Architecture Evaluation”, https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=5177
22. Carnegie Mellon University, “Architecture Tradeoff Analysis Method Collection”, https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=513908
23. COSMIC, https://cosmic-sizing.org